# Laboratorul 7 – JavaScript: stocare locală, obiecte, JSON

*Date păstrate în browser între încărcări (Local Storage și Session Storage), obiecte JavaScript și tablouri de obiecte salvate ca text JSON, introduse prin formulare în loc de prompt.*

## Obiective

- Folosirea `localStorage` și `sessionStorage` și urmărirea lor în instrumentele dezvoltatorului.
- Înțelegerea diferenței dintre cele două: cât timp și unde rămân datele.
- Lucrul cu obiecte JavaScript și transformarea lor în text și înapoi, cu `JSON.stringify` și `JSON.parse`.
- Înlocuirea ferestrelor `prompt`/`alert` cu un formular și o secțiune de rezultat în pagină.
- Publicarea lucrării pe GitHub și predarea ei prin pull request.

## Instrumente de lucru

Editor: Visual Studio Code, cu extensia Live Server. Fișierele de pornire se descarcă de la adresa `https://programarea-aplicatiilor-web.github.io/Laboratoare/L7_files.zip`. Publicarea se face în copia proprie (fork) a repository-ului `github.com/Programarea-Aplicatiilor-WEB/Laboratoare`, după pașii din pagina de instrucțiuni: `https://programarea-aplicatiilor-web.github.io/Laboratoare/instructiuni.html`. Lucrarea acestui laborator stă în folderul `<subgrupa>/<nume-prenume>/lab7/`. Datele stocate se văd cu Ctrl + Shift + I: în Chrome/Edge, tabul **Application** → **Storage** → Local storage / Session storage; în Firefox, tabul **Storage**. Acolo se pot și modifica sau șterge.

## 1. Noțiuni teoretice

**Local Storage** păstrează date în browser fără termen: rămân după închiderea filei și a browserului. **Session Storage** le păstrează doar cât timp este deschisă fila; o filă nouă pornește cu o sesiune goală. Ambele reține perechi **cheie – valoare**, iar valoarea este întotdeauna **text**.

| Construcție | Ce face |
|---|---|
| `localStorage.setItem("tip", "Dacia")` | salvează valoarea sub cheia „tip”; echivalent: `localStorage.tip = "Dacia"` |
| `localStorage.getItem("tip")` | citește valoarea; `null` dacă cheia nu există |
| `localStorage.removeItem("tip")` | șterge cheia; `localStorage.clear()` le șterge pe toate |
| `Number(localStorage.clickcount)` | transformă textul salvat în număr, ca să se poată aduna |
| `var m = {tip: "Dacia", anFab: 2012};` | obiect: un grup de proprietăți; se citesc cu `m.tip`, `m.anFab` |
| `var lista = [m1, m2];`<br>`lista.push(m3);` | tablou de obiecte; `lista.length`, `lista[0].tip` |
| `JSON.stringify(m)` | obiectul (sau tabloul) ca text: `{"tip":"Dacia","anFab":2012}` – forma în care se poate salva |
| `JSON.parse(text)` | textul JSON înapoi în obiect |
| `<form onsubmit="return salveaza()">` | formular; funcția întoarce `false`, ca pagina să nu se reîncarce la trimitere |
| `document.getElementById("tip").value` | textul scris într-un câmp `<input>` |

**Unde se păstrează:** datele aparțin site-ului, nu paginii. Toate paginile de pe același site văd aceleași chei – iar pe GitHub Pages, toate paginile de la adresa organizației (`programarea-aplicatiilor-web.github.io`) sunt un singur site. Pagina din fork-ul propriu (`<username>.github.io`) are un spațiu separat. Datele nu sunt secrete: oricine are acces la browser le vede în instrumentele dezvoltatorului.

## 2. Exerciții rezolvate

### Exercițiul 1 (rezolvat) – folderul lab7 și contoarele din L7_A_Contoare

Copiați conținutul arhivei `L7_files.zip` în subfolderul `lab7` al folderului personal și adăugați în `index.html` din folderul personal un link către `lab7/`. Deschideți apoi `L7_A_Contoare.html` și instrumentele dezvoltatorului (Ctrl + Shift + I). Apăsați de câteva ori pe fiecare buton și urmăriți valorile `clickcount` din Local storage și din Session storage în timpul folosirii, după reîncărcarea paginii, într-o filă nouă și după închiderea și redeschiderea browserului.

```
popescu-ana/
  index.html                  ← pagina personală: rând nou către lab7/
  lab7/
    index.html                ← din arhivă: butoane către cele cinci exemple
    L7_A_Contoare.html
    L7_B_User.html
    L7_C_UserPassCriptat.html
    L7_D_MasiniRaw.html       ← de modificat (ex. 3–4)
    L7_E_MasiniObj.html       ← de modificat (ex. 5–6)
```

| Situație | Contor local / Contor sesiune |
|---|---|
| reîncărcarea paginii (F5) | continuă / continuă |
| aceeași pagină, într-o filă nouă | continuă / pornește de la 1 |
| închiderea și redeschiderea browserului | continuă / pornește de la 1 |
| ștergerea cheii din instrumentele dezvoltatorului | pornește de la 1 / pornește de la 1 |

**Explicație:** valoarea se salvează ca text; de aceea codul o transformă cu `Number(…)` înainte de `+ 1` – fără ea, „1” + 1 ar da „11”. `localStorage.clickcount` și `sessionStorage.getItem("clickcount")` sunt două scrieri echivalente, folosite intenționat în exemplu. Unele browsere restaurează sesiunea la redeschiderea filelor închise, caz în care și contorul de sesiune poate continua.

### Exercițiul 2 (rezolvat) – L7_B_User și L7_C_UserPassCriptat

Studiați `L7_B_User.html`: înscrieți un utilizator, verificați-l cu parola bună și cu una greșită și urmăriți Local storage. Faceți același lucru în `L7_C_UserPassCriptat.html` și comparați sursele și valorile salvate.

| | L7_B_User / L7_C_UserPassCriptat |
|---|---|
| ce se salvează | cheia este numele utilizatorului; valoarea este parola în clar (`secret`) / amprenta MD5 a parolei (`5ebe2294ecd0…`) |
| verificarea | compară parola scrisă cu cea salvată / calculează MD5 din parola scrisă și compară amprentele |
| diferența în sursă | C încarcă biblioteca CryptoJS și folosește `CryptoJS.MD5(…)` la înscriere și la verificare |

**Observații:** MD5 nu este o criptare: amprenta nu se poate „decripta”, dar o parolă simplă se găsește ușor după amprentă în tabele publice. Paginile arată principiul; parolele reale nu se verifică în browser, ci pe server. Cele două pagini sunt pe același site, deci văd aceleași chei: un utilizator înscris în B apare „existent” și în C, dar parola lui nu se mai potrivește – în C se compară amprenta cu un text în clar. Linia a doua `<script>` din C trimite la `crypto-js.googlecode.com`, un serviciu închis; consola afișează o eroare, dar pagina funcționează, pentru că prima bibliotecă, `aes.js`, conține deja MD5. Ștergeți linia moartă.

## 3. Exerciții propuse

*Se lucrează în folderul lab7, pe fișierele L7_D și L7_E. Cele cinci câmpuri și butoane se pot așeza oricum, cu condiția să fie ușor de folosit.*

**Exercițiul 3.** Modificați fișierul `L7_D_MasiniRaw.html` astfel încât, pentru „Masina Local Storage”, datele să se introducă într-un formular (tip, model, culoare și un buton „Salvează”), nu prin `prompt`, iar mașina salvată să fie afișată în secțiunea de sub formular. La deschiderea paginii, secțiunea arată mașina deja salvată sau „Nicio mașină salvată”.

```
<div style="background-color:ForestGreen;color:white;font-size:200%">
  <p>Masina Local Storage</p>
  <form onsubmit="return salveazaMls()">
    <label for="tipL">Tip</label> <input id="tipL" required>
    <label for="modelL">Model</label> <input id="modelL" required>
    <label for="culoareL">Culoare</label> <input id="culoareL" required>
    <button type="submit">Salvează</button>
  </form>
  <div id="mls"></div>          <!-- secțiunea cu „răspunsul” -->
</div>
```

**Indicație:** scoateți `onclick` de pe `<div>` – altfel, fiecare clic într-un câmp al formularului ar porni din nou funcția. Afișarea la deschidere se face din `<body onload="…">`. Funcția apelată la `onsubmit` se termină cu `return false;`.

**Exercițiul 4.** Faceți același lucru pentru „Masina Session Storage”, cu un al doilea formular și propriile câmpuri (`id`-uri diferite). Adăugați în ambele formulare un buton „Șterge”, care șterge mașina salvată (`removeItem`). Verificați diferența dintre cele două mașini deschizând pagina într-o filă nouă. „Masina JavaScript” rămâne neschimbată.

**Exercițiul 5.** Modificați fișierul `L7_E_MasiniObj.html` astfel încât „Masina Local Storage” să fie introdusă printr-un formular cu patru câmpuri (tip, model, culoare, an de fabricație). Datele se adună într-un obiect, salvat sub o singură cheie, `masina`, cu `JSON.stringify`, și afișat în secțiunea din pagină cu `JSON.parse` – inclusiv anul. Urmăriți în Local storage forma textului salvat.

**Exercițiul 6.** Completați funcția `loadMss()`, lăsată goală în `L7_E_MasiniObj.html`, astfel încât „Masina Session Storage” să funcționeze la fel, cu formular propriu, în Session Storage. Funcțiile de afișare și de ștergere se pot scrie o singură dată, cu depozitul ca parametru: `afiseaza(localStorage, "mls")`, `afiseaza(sessionStorage, "mss")`.

**Exercițiul 7.** Publicați folderul `lab7` și pagina personală în copia proprie a repository-ului (după **Sync fork**, dacă este cazul). Verificați la adresa fork-ului fiecare pagină și urmăriți în instrumentele dezvoltatorului că datele salvate local nu apar la adresa publicată – sunt site-uri diferite. Predați lucrarea printr-un pull request cu titlul „Lab 7 – Nume Prenume – subgrupa”.

## 4. Exercițiu bonus

Creați fișierul `L7_F_Garaj.html` și adăugați-l în `lab7/index.html`: un formular adaugă mașini într-un **tablou de obiecte** salvat în Local Storage sub cheia `garaj`, iar toate mașinile apar într-un tabel, fiecare cu un buton „Șterge” propriu. Sub formular apare numărul de mașini din garaj.

**Indicații:** citiți tabloul cu `JSON.parse(localStorage.getItem("garaj"))` – sau `[]` dacă cheia nu există –, adăugați cu `push`, scoateți cu `splice(i, 1)` și salvați înapoi cu `JSON.stringify`. Tabelul se reface complet după fiecare modificare, cu o buclă `for`.